Structural Design Patterns Technical Report

Course Project: E-Commerce Order Management System (E-OMS)

1. Introduction

This report documents the structural design pattern cases identified and implemented in the E-OMS project.
The goal is to show where each required pattern solves a real design problem in the system, how the proposed design works conceptually, and why it is a suitable solution.

The required patterns covered in this report are:

- Bridge Pattern (2 cases)
- Adapter Pattern (2 cases)
- Flyweight Pattern (1 case)
- Decorator Pattern (2 cases)

For each case, the report includes:

- Problem statement from project context
- Pattern-based solution overview
- UML
- Justification and expected design benefits
- Implementation status in the Java project

---

2. Bridge Pattern

2.1 Case B1 - Notification Type vs Delivery Channel

Problem

The system sends different notification types (order confirmation, payment receipt, shipping update) through multiple channels (email, SMS, WhatsApp).
If each notification type is tightly coupled to each channel, the class hierarchy grows quickly and becomes hard to maintain.

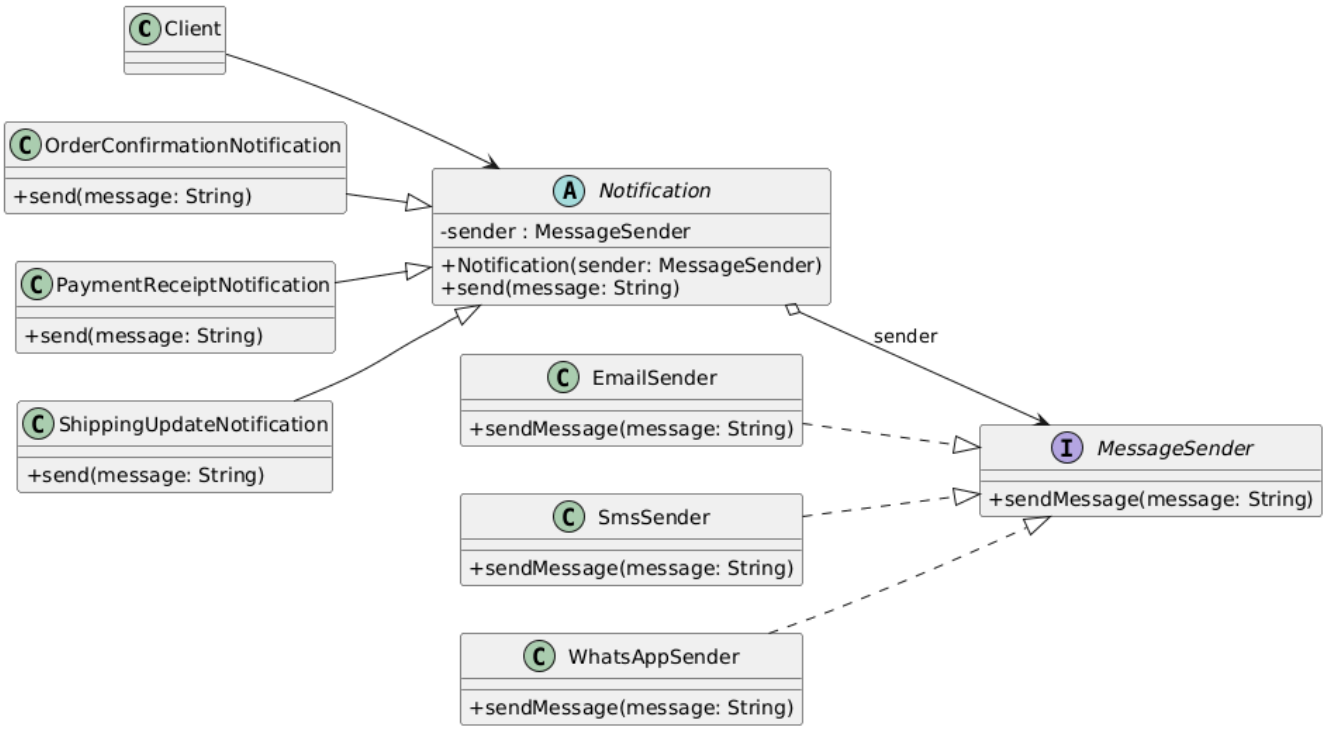
Bridge Solution

Separate:

- **Abstraction hierarchy**: `Notification` and its refined abstractions
  - `OrderConfirmationNotification`
  - `PaymentReceiptNotification`
  - `ShippingUpdateNotification`
- **Implementation hierarchy**: `MessageSender` and its implementations
  - `EmailSender`
  - `SmsSender`
  - `WhatsAppSender`

`Notification` holds a reference to `MessageSender`, allowing runtime combination of notification type and channel.

UML



Justification

Bridge is appropriate because both dimensions can vary independently:

- New notification types can be added without changing sender classes.
- New channels can be added without changing notification abstractions.
- Runtime wiring in `EomsApplication` selects suitable channels per use case.

Implementation Status

Implemented in package `com.eoms.bridge_notification` and wired in `com.eoms.app.EomsApplication`.

2.2 Case B2 - Report Type vs Output Format

Problem

The admin module generates different report types (e.g., sales, inventory) and may require different output formats (e.g., PDF, Excel).
Without separation, each report type may need duplicate formatting logic.

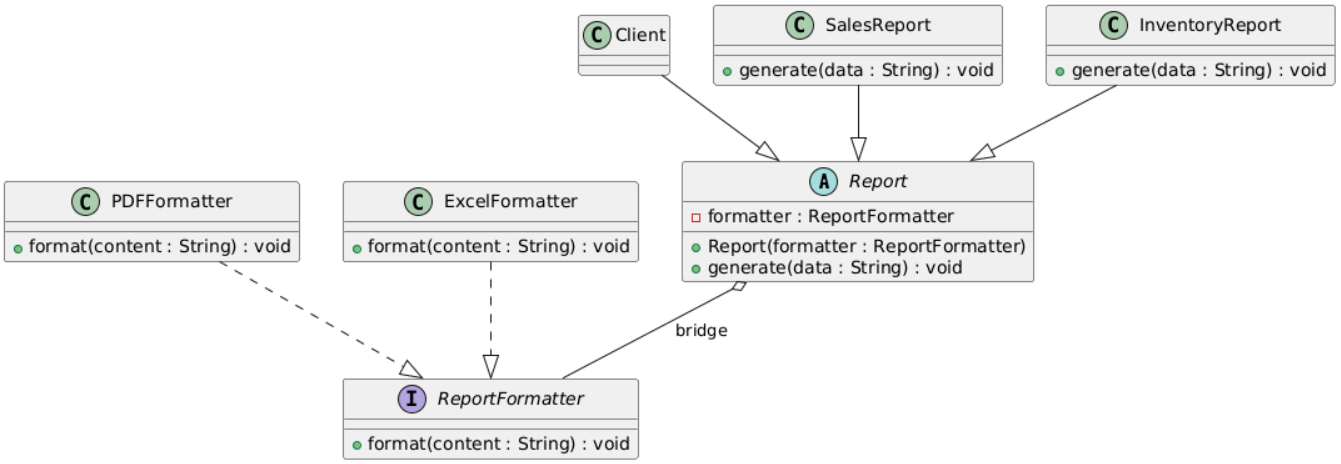
Bridge Solution

Separate:

- **Abstraction hierarchy**: `Report` and refined reports
  - `SalesReport`
  - `InventoryReport`
- **Implementation hierarchy**: `ReportFormatter`
  - `PDFFormatter`
  - `ExcelFormatter`

Each report delegates formatting to a `ReportFormatter`, enabling flexible combinations.

UML



Justification

Bridge avoids combinational class explosion and enforces separation of concerns:

- Report content logic stays in report classes.
- Rendering/export logic stays in formatter classes.
- New report type or format can be introduced independently.

Implementation Status

Implemented in package `com.eoms.bridge_report`.

---

3. Adapter Pattern

3.1 Case A1 - Internal Payment Processor to External Stripe Gateway

Problem

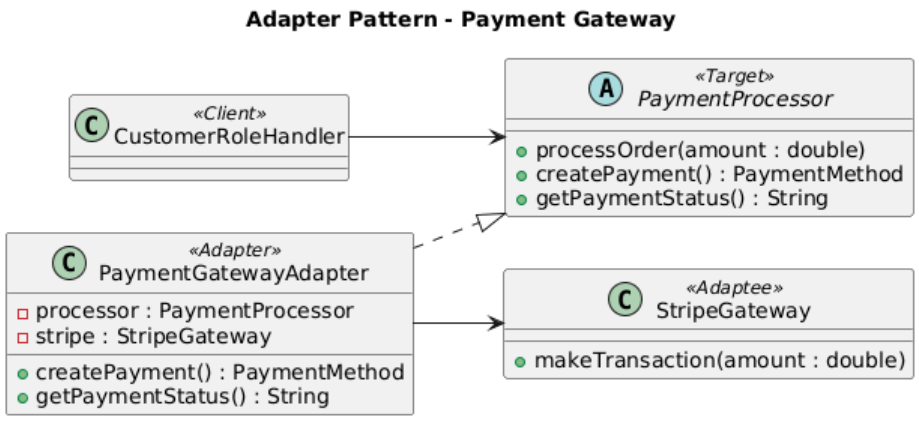
The project uses internal payment processing abstractions (`PaymentProcessor` , `PaymentMethod`), while external payment transaction calls are made through `StripeGateway` , which has a different API and contract.

Adapter Solution

`PaymentGatewayAdapter` adapts internal payment processor behavior to the external Stripe API:

- Delegates internal payment execution first
- Then invokes `StripeGateway` transaction call
- Exposes a compatible `PaymentProcessor` interface to the rest of the system

UML



Justification

Adapter is suitable because it resolves interface mismatch while preserving existing client code:

- Customer/payment flow still depends on `PaymentProcessor`.
- External service details remain encapsulated in adapter.
- Reduces coupling to third-party API shape changes.

Implementation Status

Implemented as `com.eoms.adapter.PaymentGatewayAdapter`.

3.2 Case A2 - Shipping Service to DHL API

Problem

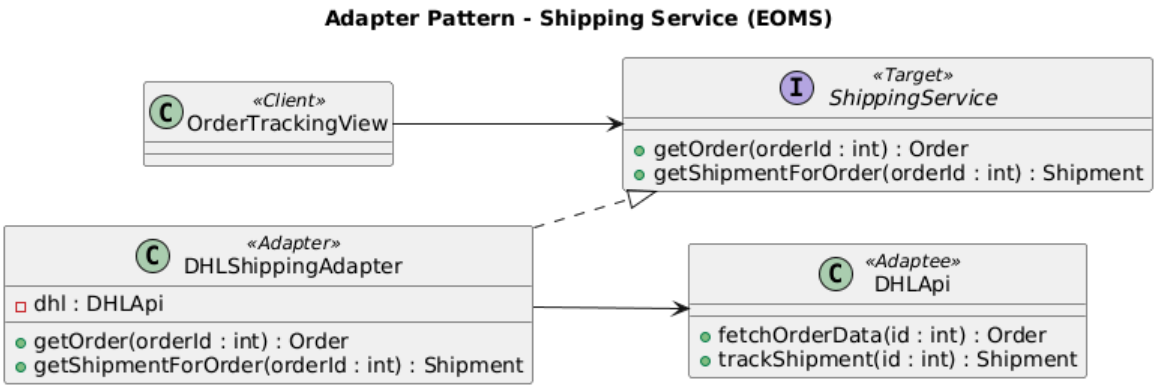
Order tracking flow expects the project-level shipping abstraction (`ShippingService`), but shipping data operations are performed through external `DHLApi` methods with a different interface.

Adapter Solution

`DHLShippingAdapter` maps `ShippingService` operations to calls of `DHLApi`:

- Retrieves order/shipment data in terms expected by the project
- Hides external API method signatures from the UI/service layers

UML



Justification

Adapter provides interoperability and isolation:

- Existing tracking view code depends only on `ShippingService`.
- External API changes are localized in adapter.
- Maintains clean architecture boundaries.

Implementation Status

Implemented as `com.eoms.adapter.DHLShippingAdapter`.

---

4. Flyweight Pattern

4.1 Case F1 - Shared Product Type State

Problem

The product catalog may contain many product instances where product type values (`PHYSICAL`, `DIGITAL`, `SERVICE`) repeat frequently.
Storing repeated intrinsic type-related state per object is unnecessary overhead.

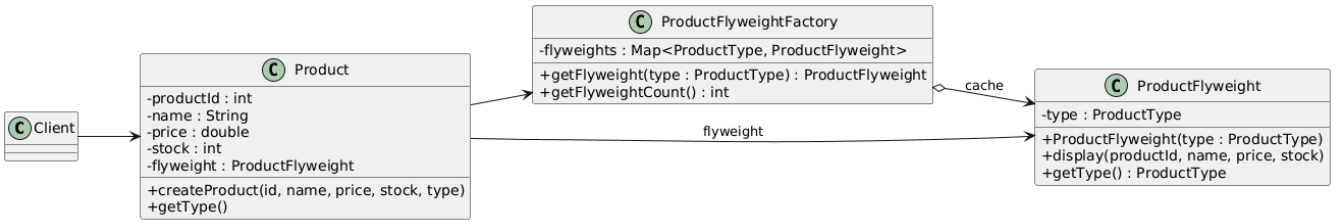
Flyweight Solution

Use `ProductFlyweightFactory` to cache and reuse `ProductFlyweight` objects by `ProductType`:

- **Intrinsic (shared) state**: product type
- **Extrinsic (per-instance) state**: product id, name, price, stock

`Product` requests flyweights from factory instead of creating duplicate intrinsic objects.

UML



Justification

Flyweight is beneficial when many objects share identical intrinsic state:

- Reduces object duplication in memory
- Centralizes shared state creation and reuse
- Keeps instance-specific data external to flyweight

Implementation Status

Implemented in package `com.eoms.flyweight` and used by `com.eoms.entity.Product`.

---

5. Decorator Pattern

5.1 Case D1 - Timestamped Notifications

Problem

Notification channels need optional additional behavior (timestamp metadata) without modifying all sender classes directly.

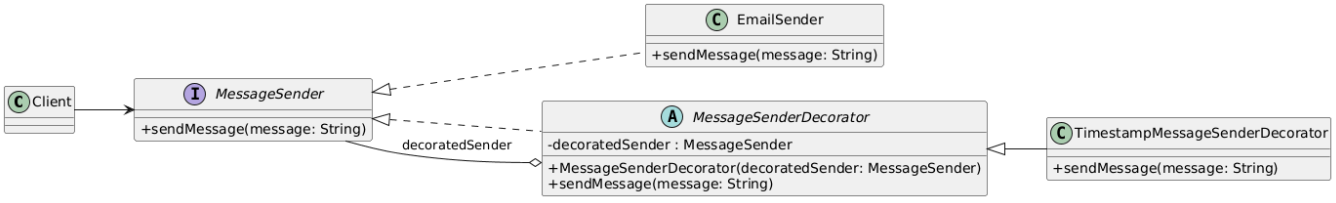
Decorator Solution

Decorate `MessageSender` using:

- Base decorator: `MessageSenderDecorator`
- Concrete decorator: `TimestampMessageSenderDecorator`

This adds timestamp behavior before delegating to wrapped sender.

UML



Justification

Decorator is appropriate for dynamic behavior extension:

- No changes required in core senders (`EmailSender`, `SmsSender`, `WhatsAppSender`)
- Behavior can be attached at runtime in composition root (`EomsApplication`)

- Supports open/closed principle for sender enhancements

Implementation Status

Implemented in package `com.eoms.bridge_notification.decorator`; applied to email, WhatsApp, and SMS channels.

5.2 Case D2 - COD Maximum Amount Rule

Problem

Cash On Delivery should be blocked for high-value orders (cap = 2000), but base payment method classes should remain unchanged.

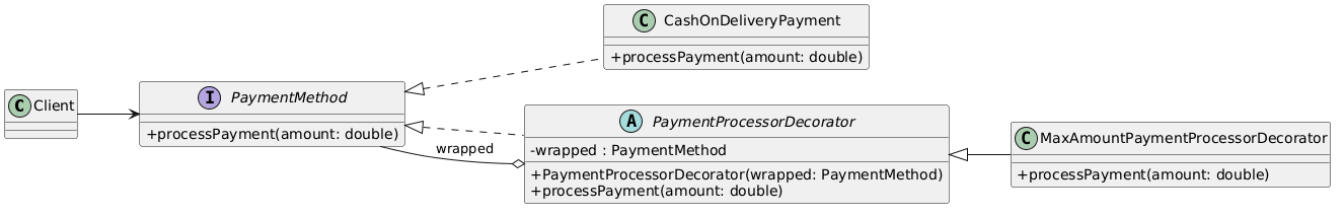
Decorator Solution

Decorate `PaymentMethod` using:

- Base decorator: `PaymentProcessorDecorator` (abstract)
- Concrete decorator: `MaxAmountPaymentProcessorDecorator`

The decorator checks amount; if above cap, it logs decline and stops processing.

UML Placeholder



Justification

Decorator cleanly adds policy behavior around payment execution:

- Reuses existing `CashOnDeliveryPayment` without editing it
- Keeps rule isolated and composable
- Enables future stacked decorators (e.g., logging, validation, retry) if needed

Implementation Status

Implemented in package `com.eoms.factory.decorator`; used in payment processor provisioning flow.

---

6. Summary and Conclusion

The selected structural patterns solve concrete design and integration problems in E-OMS:

- **Bridge** decouples two varying dimensions (notification type/channel, report type/format).
- **Adapter** integrates external APIs while preserving internal interfaces.
- **Flyweight** shares repeated intrinsic product state efficiently.
- **Decorator** adds optional behaviors and business constraints without changing base components.

Github link: https://github.com/Kiro-create/NexOrder